

ChatGPT



```
class Solution {  
    public int[] insertionSort(int[] nums) {  
        int n=nums.length;  
        for(int i=0;i<=n-1;i++){  
            int j=i;  
            while(j>0 && nums[j-1]>nums[j]){  
                int temp = nums[j-1];  
                nums[j-1] = nums[j];  
                nums[j] = temp;  
                j--;  
            }  
        }  
        return nums;  
    }  
}
```

teach me insertion sort
Show moreShow less

Absolutely. Your code is a **correct insertion sort** implementation. Let's understand *why* it works rather than just memorizing it.

1. The basic idea

Imagine sorting playing cards in your hand:

- You start with one card → already sorted.
- Pick the next card.
- Compare it with cards to its left.
- Move bigger cards one position to the right.
- Put the picked card into its correct position.

Insertion sort does exactly this with an array.

Start with one sorted value

Pass 1 of 6-1 sorted

Sorted prefix

7 3 8 2 6 4 5

1234567

Insertion step

ShufflePlayStepFinish pass

Give feedback

For example:

[5, 3, 4, 1, 2]

We consider the left side as **sorted** and the right side as **unsorted**:

[5 | 3, 4, 1, 2]
↑
sorted

Take 3:

[5 | 3, 4, 1, 2]
↑
key

Since $5 > 3$, swap/move:

[3, 5 | 4, 1, 2]

Now take 4:

[3, 5 | 4, 1, 2]
↑

$5 > 4$, so move 5 right:

[3, 4, 5 | 1, 2]

Continue until:

[1, 2, 3, 4, 5]

2. Now your code

You have:

Java

```
for(int i=0; i<=n-1; i++){
    int j=i;

    while(j>0 && nums[j-1]>nums[j]){
        int temp = nums[j-1];
        nums[j-1] = nums[j];
        nums[j] = temp;
        j--;
    }
}
```

Let's break it down.

i chooses the element

Java

```
for(int i=0; i<=n-1; i++)
```

i moves from left to right.

For:

```
[5, 3, 4, 1, 2]
```

i will be:

```
i = 0 → 5
i = 1 → 3
i = 2 → 4
i = 3 → 1
i = 4 → 2
```

The important idea is:

Before processing index **i**, everything before **i** is already sorted.

3. Why j = i?

Java

```
int j = i;
```

j is the element we're currently trying to insert into the sorted portion.

Suppose:

```
[3, 5, 4, 1, 2]
      ↑
     i=2
```

We want to insert **4** into:

```
[3, 5]
```

So:

```
Java
j = 2;
```

4. The most important line

```
Java
while(j > 0 && nums[j-1] > nums[j])
```

There are **two conditions**.

Condition 1

```
Java
j > 0
```

This means:

Don't go outside the beginning of the array.

Condition 2

```
Java
nums[j-1] > nums[j]
```

This means:

The element immediately before me is bigger than me, so I need to move left.

For:

```
[3, 5, 4]
  ↑  ↑
  j-1 j
```

We have:

```
5 > 4
```

Therefore, swap them:

```
[3, 4, 5]
```

Then:

```
Java
j--;
```

Now **j** points one position further left.

5. Let's trace your code

Take:

```
[5, 3, 4, 1, 2]
```

i = 0

```
[5 | 3, 4, 1, 2]
```

Nothing to do because $j = 0$.

i = 1

```
[5 | 3, 4, 1, 2]
    ↑
    j
```

Check:

```
5 > 3
```

Yes → swap:

```
[3, 5 | 4, 1, 2]
```

j--

Now $j = 0$, stop.

i = 2

```
[3, 5 | 4, 1, 2]
      ↑
```

Check:

```
5 > 4
```

Swap:

```
[3, 4, 5 | 1, 2]
```

Now $j = 1$.

Check again:

```
3 > 4
```

False.

Stop.

i = 3

```
[3, 4, 5 | 1, 2]
           ↑
```

5 > 1 → swap:

```
[3, 4, 1, 5, 2]
```

4 > 1 → swap:

```
[3, 1, 4, 5, 2]
```

3 > 1 → swap:

```
[1, 3, 4, 5, 2]
```

Now j = 0, stop.

i = 4

```
[1, 3, 4, 5 | 2]
              ↑
```

Move 2 left:

```
[1, 3, 4, 2, 5]
[1, 3, 2, 4, 5]
[1, 2, 3, 4, 5]
```

Done.

6. The key thing to remember

Think of insertion sort as maintaining this invariant:

```
[ SORTED | UNSORTED ]
```

Every iteration takes **one element from the unsorted part** and inserts it into the correct position in the sorted part.

For example:

```
[5 | 3 4 1 2]
[3 5 | 4 1 2]
[3 4 5 | 1 2]
[1 3 4 5 | 2]
[1 2 3 4 5 | ]
```

That's insertion sort.

7. Your swap is doing the "shifting"

Your code uses:

```
Java
int temp = nums[j-1];
nums[j-1] = nums[j];
nums[j] = temp;
```

This swaps adjacent elements.

So:

```
[3, 5, 4]
  ↑
```

becomes:

```
[3, 4, 5]
```

One important distinction: **classic insertion sort is often written using shifting instead of repeated swapping**, but your version is still insertion sort and is perfectly valid.

8. Complexity

For an array of n elements:

Case	Time
Already sorted	$O(n)$
Average	$O(n^2)$
Reverse sorted	$O(n^2)$
Extra space	$O(1)$

Why $O(n^2)$?

In the worst case, each new element may need to travel all the way to the beginning:

```
[5, 4, 3, 2, 1]
```

So you can get roughly:

```
1 + 2 + 3 + ... + (n-1)
```

comparisons/swaps $\rightarrow O(n^2)$.

The one sentence to memorize

Insertion sort takes each element and moves it left until it reaches its correct position in the already-sorted left portion.

Your implementation is doing exactly that.



complete java code for it with main function

Compare with other AI ⚡

Sure — here is the complete Java program using your exact insertion-sort approach:

Java

```
import java.util.Arrays;

class Solution {

    public int[] insertionSort(int[] nums) {
        int n = nums.length;

        for (int i = 0; i <= n - 1; i++) {
            int j = i;

            while (j > 0 && nums[j - 1] > nums[j]) {

                int temp = nums[j - 1];
                nums[j - 1] = nums[j];
                nums[j] = temp;

                j--;
            }
        }

        return nums;
    }

    public static void main(String[] args) {

        int[] nums = {5, 3, 4, 1, 2};

        Solution obj = new Solution();

        int[] result = obj.insertionSort(nums);

        System.out.println("Sorted array: " + Arrays.toString(result));
    }
}
```

Output

Sorted array: [1, 2, 3, 4, 5]

One small improvement

This:

Java

```
for (int i = 0; i <= n - 1; i++)
```

can more commonly be written as:

Java

```
for (int i = 0; i < n; i++)
```

So I'd recommend:

Java

```
for (int i = 0; i < n; i++)
```

Both are correct.

ChatGPT can make mistakes. Check important info.